

COMP2113 Programming Technologies

Assignment 1

Deadline: 1 March (Sunday), 2026 23:59

General Instructions

Submit your assignment via VPL on Moodle. Ensure that your program can execute, and generate the required outputs in VPL. Work incompatible with the VPL may not be marked.

For shell scripts (Problem 1 and 2), they must start with the header `#!/bin/bash`, and will be executed using the Bash shell on our standard environment.

As a developer, ensure that your code works flawlessly in the intended environment, not just your own. While you may develop your work in your own environment, always test your program in our standard environment before submission.

Evaluation

For tasks requiring **user input**, utilize the **standard input**. Likewise, your program should **output/print** through the **standard output**. Strict adherence to the sample output format is required, or your answer may be marked incorrect.

Your code will be automatically graded for technical correctness. Essentially, we use test cases to evaluate your solution, failure to pass any of the test cases may result in zero marks. Partial credits are generally not given for incomplete solutions as it may be challenging to objectively assess incomplete program logic. However, your work may still be considered on a case-by-case basis during the rebuttal stage.

Additional test case will be used during grading. Scoring full marks on VPL does not ensure full marks in the assignment. Sample test cases may or may not encompass all boundary cases. Designing proper test cases to verify your program's accuracy is part of the assessment.

Academic dishonesty

Your code will be cross-checked with other submissions and online sources for logical duplication. Note that providing your work to others, aiding others in copying, or copying from others will be considered plagiarism, and will be dealt with as per departmental policy. Please refer to the course information notes for more details.

Use of generative AI tools, like ChatGPT, is not permitted for all assignment.

Getting help

You are not alone! If you are stuck, post your query on the course forum. This assignment should be educational and rewarding, not frustrating. We are here to help, but we can only do so if you reach out.

Please avoid spoilers on the discussion forum. Do not post any code directly related to the assignments. You are, however, encouraged to discuss general concepts on the forums.

Submission

Deadlines are strictly enforced. Resubmission beyond the submission period will not be accepted.

Late Policy:

- If you submit within 2 days after the deadline, 30% deduction.
- If you submit within 3-5 days after the deadline, 50% deduction.
- After that, no mark.

Problem 1: Log File Analyzer

Write a shell script named `1.sh` that analyzes a web server access log and prints a summary report.

Input:

Command-Line Input

The script is executed as:

```
./1.sh <logfile>
```

Log File Format

Except the first line, each line contains exactly 5 space-separated fields:

```
<IP_ADDRESS> <TIMESTAMP> <HTTP_METHOD> <ENDPOINT> <STATUS_CODE>
```

Example

```
192.168.1.1 2025-01-10T10:00:01 GET /index.html 200
```

Output:

The script must print exactly the following lines (to the standard output), in this order:

```
Total requests: <N>
Successful requests: <N> (status code 200-299)
Client errors: <N> (status code 400-499)
Server errors: <N> (status code 500-599)
Top 3 endpoints:
<endpoint> <count>
<endpoint> <count>
<endpoint> <count>
```

Formatting Rules

- All labels must match exactly (case-sensitive).
- Use one space between words and numbers.
- The "Top 3 endpoints:" line must always be printed.
- Print at most 3 endpoints, in the reverse order of <count>.
- Each endpoint must appear on its own line.

Assumptions:

- The log file contains only valid lines.
- No header row is present.
- Status codes are integers between 100-599.
- No endpoint has the same <count>.

Sample Test Cases:

1_1

Input file

```
10.0.0.1 2025-01-01T12:00:00 GET /home 200
10.0.0.2 2025-01-01T12:00:01 GET /home 200
10.0.0.3 2025-01-01T12:00:02 POST /login 403
```

Expected output

```
Total requests: 3
Successful requests: 2
Client errors: 1
Server errors: 0
Top 3 endpoints:
/home 2
/login 1
```

1_2

Input file

```
192.168.0.1 2025-02-01T09:00:00 GET /login 401
192.168.0.2 2025-02-01T09:00:01 POST /login 403
192.168.0.3 2025-02-01T09:00:02 POST /login 200
192.168.0.4 2025-02-01T09:00:03 GET /profile 404
192.168.0.5 2025-02-01T09:00:04 GET /settings 400
192.168.0.6 2025-02-01T09:00:05 GET /profile 200
```

Expected output

```
Total requests: 6
Successful requests: 2
Client errors: 4
Server errors: 0
Top 3 endpoints:
/login 3
/profile 2
/settings 1
```

1_3

Input file

(empty)

Expected output

Total requests: 0
Successful requests: 0
Client errors: 0
Server errors: 0
Top 3 endpoints:

Problem 2: Count ingredient

Write a shell script named 2.sh to count the number of dishes that contain a given ingredient.

Input:

Command-Line Input

The script is executed as:

```
./2.sh <recipe_file>
```

Recipe File Format

The first line specifies the given ingredient
From the second line, it looks like:

```
<dish_name>: <ingredient1>, <ingredient2>, ...
```

Example

```
sugar  
Pancakes: flour, milk, sugar, butter
```

Output:

The script must print the following line with <number> determined by recipes:

```
The number of dishes that contain the given ingredient: <number>
```

Assumptions:

- Except the first line, each dish line contains exactly one colon (:) separating the dish name and ingredients.
- Ingredients are separated by commas (,).
- A dish is counted at most once, even if multiple ingredients in that dish match.
- Ingredient names do not contain commas.
- We consider it a match as long as one ingredient of a dish contain the word of the given ingredient, e.g., apple juice and apple are considered a match, apples and apple are considered as a match.

Sample Test Cases:

2_1

Input file

```
sugar  
Pancakes: flour, milk, sugar, butter  
Salad: lettuce, olive oil, salt  
Cake: flour, brown sugar, eggs
```

Expected output

```
The number of dishes that contain the given ingredient: 2
```

2_2

Input file

oil
Omelette: eggs, milk, cheese
Soup: water, salt, pork

Expected output

The number of dishes that contain the given ingredient: 0

2_3

Input file

chili
Beef Chili: ground beef, chili powder, tomato sauce, kidney beans, onions

Expected output

The number of dishes that contain the given ingredient: 1